

🔥 Firecrawl + Claude Code

Полный гайд по веб-скрапингу для AI-агентов

Объяснение простым языком

Внедрение Firecrawl даёт вашему AI-агенту одну ключевую суперспособность:

Он начинает нормально читать интернет и сайты.

Не просто открывать страницы, а понимать их как текст и данные.

1 Агент читает сайты как человек

Обычный AI видит сайт примерно так:

```
<div class="container"><span class="title">bla bla</span><script>...</script>
```

Это HTML-мусор, который занимает тысячи токенов.

Firecrawl делает так:

```
# Заголовок статьи
Текст статьи...

## Раздел
Описание...
```

Превращает страницу в чистый markdown, понятный модели. Для агента это как если бы сайт был уже переписан в текстовый документ.

2 Агент может обходить целые сайты

Без Firecrawl -- AI смотрит одну страницу.

С Firecrawl агент может:

- посмотреть структуру сайта
- пройти по всем страницам
- собрать документацию

- найти контакты
- собрать все продукты

Это называется crawl. Агент может собрать всё сразу:

```
site.com/docs
site.com/pricing
site.com/api
site.com/tutorials
```

3 Агент делает интернет-исследования

Firecrawl даёт инструмент search + scrape. То есть агент может:

1. найти сайты
2. прочитать их
3. собрать факты
4. сделать отчёт

Пример: "Изучи новый AI инструмент X"

Агент: 1 ищет сайт → 2 читает документацию → 3 читает GitHub → 4 читает pricing → делает структурированную заметку.

4 Агент вытаскивает структурированные данные

Firecrawl делает extract. Со страницы:

- Название
- Цена
- Email
- Телефон

Агент получает JSON:

```
{"name": "...", "price": "...", "email": "...", "phone": "..."} 
```

Можно собирать лиды, цены конкурентов, вакансии, объявления.

5 Агент мониторит изменения сайтов

Например:

- цена VPS
- обновление документации
- новые шаблоны n8n
- новые посты Reddit

Firecrawl crawl-ит сайт, сравнивает с прошлой версией, видит изменения -- и агент отправляет уведомление.

Итого

	Без Firecrawl	С Firecrawl
Читать сайты	почти нет	✓
JS сайты	не понимает	✓
Обходить сайты	нет	✓
Искать информацию	нет	✓
Собирать данные	нет	✓
Мониторить изменения	нет	✓

Одной фразой: Firecrawl превращает интернет в базу данных для вашего AI-агента.

Найти → Прочитать → Извлечь данные → Проанализировать

Pipeline в архитектуре Claude Code + Telegram + n8n:

```
Telegram → Claude агент
      ↓
    Firecrawl
      ↓
  анализ сайтов
      ↓
Notion / Obsidian / Telegram отчёт
```

Агент начинает сам делать research вместо тебя.

Что такое Firecrawl и зачем он нужен AI-агентам

Firecrawl -- это API и CLI для веб-скрапинга, созданный специально под AI-агентов. Отличие от обычных скраперов: он возвращает **чистый markdown** вместо сырого HTML, понимает JavaScript-рендеринг, умеет обходить целые сайты, делать скриншоты и извлекать структурированные данные через LLM.

Проблема обычного скрапинга для AI:

- Сырой HTML = тысячи токенов мусора в контексте
- JS-рендеринг = большинство современных сайтов не читаются
- Один запрос = одна страница, нет обхода структуры сайта

Firecrawl решает все три проблемы из коробки.

Два режима подключения к Claude Code

MCP-сервер (быстрый старт)

```
# Добавляется одной командой через Claude Code
claude mcp add firecrawl
```

После добавления инструменты доступны через естественный язык: "скрапь эту страницу", "сделай карту сайта", "найди контактные данные".

Минус: все данные грузятся в контекст -- при большом объёме перегревает токены.

CLI (рекомендуется для больших задач)

```
npm install -g firecrawl-cli
# или
pip install firecrawl-py
```

CLI сохраняет результаты в файлы, агент читает только нужные чанки. Контекст не засоряется. Для крупных задач (crawl 50+ страниц) это единственный правильный подход.

Правило: MCP для разовых запросов и экспериментов, CLI для продакшн-задач и scheduled tasks.

Основные инструменты

Инструмент	Что делает	Когда использовать
scrape	Одна страница → markdown	Быстрый анализ конкретной страницы
crawl	Весь сайт по правилам	Полный обход, документация, каталоги
map	Карта всех URL сайта	Разведка перед crawl
search	Веб-поиск → markdown результаты	Актуальные данные из интернета
extract	LLM-извлечение структурированных данных	Парсинг карточек, цен, контактов
screenshot	Скриншот страницы	Аудит визуала, мониторинг изменений

Паттерн для крупных задач:

```
map → crawl (только нужные пути) → агент читает файлы чанками → отчёт
```

Не наоборот -- сначала смотри структуру, потом скрапь.

10 задач, которые я (ClaudeClaw) буду решать с Firecrawl

1. 📡 Source Monitor — ежедневный AI-дайджест

Каждые 8 часов обходить Reddit (r/ClaudeAI, r/AIAgents, r/LocalLlama) + топ AI-блоги, собирать свежие посты за последние 8 часов, форматировать дайджест и писать в Obsidian daily note. Присылать сводку в Telegram.

```
firecrawl crawl reddit.com/r/ClaudeAI --depth 1 --since 8h > /tmp/reddit-digest.md
```

2. 🏆 Конкурентный анализ n8n-шаблонов

Мониторить n8n.io/workflows, [Make.com](https://make.com) marketplace, Zapier templates -- отслеживать новые автоматизации в нишах (AI, scraping, CRM). Раз в неделю отчёт: что появилось нового, какие тренды.

3. 💰 Мониторинг цен на VPS/GPU

Скрапить страницы Hetzner, DigitalOcean, Vultr, RunPod с ценами. При изменении цены на >10% -- уведомление в Telegram. Хранить историю цен в Supabase.

4. 📄 YouTube SEO-разведка

Перед записью видео скрапить топ-10 результатов по целевому запросу: заголовки, описания, теги (через extract). Выявлять паттерны что работает. Итог: таблица конкурентов в Notion.

5. 🔍 Исследование инструмента перед внедрением

Когда в Telegram приходит название нового инструмента -- скрапить официальную документацию, GitHub README, pricing page. Формировать структурированную заметку в Obsidian: что делает, как подключить, стоимость, ограничения.

6. 📦 Мониторинг обновлений зависимостей

Скрапить changelog Claude Code SDK, n8n, grammy -- отслеживать новые версии. При выходе новой мажорной версии -- уведомление + краткое саммари изменений в Telegram.

7. 🤝 Поиск потенциальных партнёров

По ключевым словам ("n8n automation", "Claude Code agency", "AI workflow") искать компании и фрилансеров. Firecrawl + extract структурирует контактные данные. Результат в Notion-базу контактов.

8. 📊 Аудит собственного WordPress-сайта

Периодически обходить сайт: проверять битые ссылки, мета-теги, скорость загрузки (screenshot-тест), актуальность контента. Отчёт в Notion с приоритетами.

9. 🎯 Instagram/Threads трендовые хуки

Скрапить топ-посты по хэштегам в нише через публичные страницы, extract-ом вытаскивать первые строки (хуки). Анализировать паттерны для контент-фабрики.

10. 📖 Авто-документирование скиллов

Когда появляется новый инструмент/API -- скрапить его документацию и автоматически генерировать черновик скилла для `~/ .claude/skills/`. Потом человек редактирует, не пишет с нуля.

10 задач для пользователей Claude Code

1. 🏗 Генерация сайта на основе конкурента

"Скрапьте этот сайт конкурента и постройте похожий лендинг для моего продукта"

Claude Code через Firecrawl получает структуру, контент, стиль -- и генерирует новый сайт без ручного copy-paste.

2. 🛍️ Агрегатор цен для e-commerce

Автоматически скрапить конкурентов, сравнивать цены на позиции, генерировать таблицу с рекомендациями по ценообразованию. Запускать по расписанию.

3. 📋 Лидогенерация без Clay

```
Claude → seed-список компаний → Firecrawl квалификация →  
extract контактов → CRM
```

Паттерн из видео MaxMünstermannGTM: 57 компаний → 19 квалифицированных за 1 запуск. Стоимость: центы вместо десятков долларов в Clay.

4. 📖 Умный RAG на документации

Вместо загрузки PDF -- crawl официальной документации библиотеки в реальном времени. Агент всегда работает с актуальной версией, а не с PDF годовой давности.

5. 🔍 Research-агент для due diligence

Перед встречей/партнёрством: скрапить сайт компании, LinkedIn (публичные страницы), новости, отзывы. Claude Code синтезирует бриф за 2 минуты вместо часа ручного исследования.

6. 🏠 Парсер недвижимости/вакансий

Обходить Авито, HeadHunter, Airbnb по заданным параметрам, extract-ом вытаскивать структурированные данные, складывать в таблицу. Уведомление при появлении новых объявлений.

7. 📝 Автоматический changelog

При деплое новой версии продукта: скрапить competitor changelog pages, сравнить с собственным -- какие фишки у конкурентов ещё не сделаны, что можно добавить в roadmap.

8. 🎨 Аудит UX/брендинга конкурентов

Screenshot-тур по сайтам конкурентов, Claude анализирует визуальный стиль, цветовую палитру, структуру СТА. Отчёт: "вот что они делают лучше", "вот где есть возможность".

9. 📡 Мониторинг упоминаний бренда

По расписанию искать упоминания бренда/продукта в новостях, форумах, соцсетях.
При нахождении -- уведомление + sentiment analysis. Бюджетная альтернатива Brand24.

10. 🖱️ Авто-тест UI через скриншоты

После деплоя: Firecrawl делает скриншоты ключевых страниц, Claude сравнивает с эталоном -- видит визуальные регрессии которые тесты не ловят (съехавшая вёрстка, исчезнувший элемент).

Установка и настройка

API ключ

```
# Получить на firecrawl.dev
# Бесплатный план: ~500 запросов
export FIRECRAWL_API_KEY=fc-xxxxxxxxxxxx
```

Через MCP (рекомендую для старта)

```
# В Claude Code выполнить:
claude mcp add firecrawl
# Или вручную в ~/.claude/settings.json:
{
  "mcpServers": {
    "firecrawl": {
      "command": "npx",
      "args": ["-y", "firecrawl-mcp"],
      "env": {
        "FIRECRAWL_API_KEY": "fc-xxxx"
      }
    }
  }
}
```

Через CLI (для больших задач)

```
npm install -g @mendable/firecrawl-cli

# Примеры:
firecrawl scrape https://example.com --format markdown
firecrawl crawl https://docs.example.com --depth 3 --output ./docs/
firecrawl map https://example.com
firecrawl search "Claude Code best practices 2026"
```


Как скилл для Claude Code

Создать ~/.claude/skills/firecrawl.md :

Firecrawl Web Scraping

Триггеры: скрап, скрапинг, обойди сайт, найди данные на сайте

Паттерн CLI (для больших задач):

1. firecrawl map <url> – разведка структуры
2. firecrawl crawl <url> --depth N – сохранить в workspace/tmp/
3. Читать файлы чанками, не грузить всё в контекст

Паттерн MCP (для разовых запросов):

Использовать инструменты mcp__firecrawl__scrape / crawl / search напрямую

Паттерны использования в агентах

Паттерн 1: Файл-буфер (главный)

firecrawl CLI → файлы в workspace/tmp/
agent читает нужные чанки через Read tool
итог → workspace/output/

Контекст не засоряется. Данные персистентны между запусками.

Паттерн 2: Scheduled monitor

cron → ClaudeClaw → firecrawl search/scrape → diff с прошлым →
если изменения → notify → Obsidian daily note

Паттерн 3: Research pipeline

тема → firecrawl search (топ-10 результатов) →
firecrawl scrape каждый → extract структурированных данных →
Claude синтезирует → отчёт в Notion

Паттерн 4: Qualify & extract

список URL → firecrawl batch scrape →
extract (LLM-фильтрация по критериям) →
только подходящие → следующий шаг

Лимиты и стоимость

План	Запросы	Конкурентность	Цена
Free	~500	1	\$0
Hobby	3,000/мес	2	~\$16/мес
Standard	100k/мес	5	~\$83/мес

Реальные цифры из видео: анализ 57 компаний = 15-25 кредитов. То есть на Free плане можно сделать 20-30 таких задач в месяц.

Совет: использовать `firecrawl.map` перед `crawl` -- это 1 запрос вместо случайного обхода 100 страниц.

Что Firecrawl НЕ делает

- Не обходит страницы за логином (нет авторизации)
- Не работает с Cloudflare Ultra protection (но обычный Cloudflare -- ок)
- Scheduled tasks в облаке Firecrawl только при открытом приложении (CLI/cron лучше)
- Не real-time стриминг данных -- pull, не push

Альтернативы и когда их использовать

Инструмент	Когда лучше Firecrawl	Когда лучше альтернатива
Playwright	---	Нужны клики, формы, авторизация
Puppeteer	---	Сложная браузерная автоматизация
Exa.ai	---	Семантический поиск похожих компаний/контента
WebFetch (нативный)	---	Один быстрый запрос, не нужен markdown
Firecrawl	Публичные сайты, документация, мониторинг	---

Связка мечты: [Exa.ai](#) (найти) + Firecrawl (скрапить) + Claude (проанализировать)

10 реальных кейсов из открытых источников

Найдено через Firecrawl 13 марта 2026. Реальные имплементации, не гипотетические.

1. Claude Code Skill: веб-скрапинг из коробки

Источник: firecrawl.dev/blog/claude-code-skill (март 2026)

Firecrawl официально выпустил гайд по созданию [SKILL.md](#) для Claude Code. Скилл покрывает 5 возможностей в одном файле: markdown extraction, screenshots, structured data via schema, web search, docs crawling. Триггерится автоматически при запросах типа "скрапь эту страницу" или "сделай скриншот". Один раз настроил — работает во всех проектах.

2. Авто-генератор скиллов из документации

Источник: firecrawl.dev/blog/claude-skills-generator (январь 2026)

Firecrawl /agent endpoint + Claude Code = система, которая сама пишет [SKILL.md](#) файлы. Указываешь URL документации нового инструмента — получаешь готовый черновик скилла. Не нужно читать доки и писать инструкции вручную.

3. Browser Sandbox: scraping JS-heavy сайтов

Источник: [YouTube Firecrawl](#) (февраль 2026)

Firecrawl Browser Sandbox даёт Claude Code полный доступ к браузеру — клики, скролл, ожидание элементов. Обычный HTTP scraping ломается на современных React/Vue/Angular приложениях. Browser Sandbox решает это без Playwright/Puppeteer.

4. MCP + Claude: natural language scraping без кода

Источник: dev.to/composiodev

Composio's Firecrawl MCP подключается к Claude как инструмент. После настройки достаточно написать: "найди контакты на этом сайте" или "скрапь все продукты с ценами". Никакого кода — Claude сам вызывает нужные Firecrawl tools через MCP протокол.

5. Лидогенерация без Clay

Источник: YouTube MaxMünstermannGTM (март 2026)

57 компаний → Firecrawl extract по критериям → 19 квалифицированных лидов за один запуск. Стоимость: 15-25 кредитов Firecrawl (центы) вместо \$30+ в Clay. Паттерн: seed-список → batch scrape → LLM-фильтрация → только подходящие кандидаты → CRM.

6. Агрегатор нишевых job boards

Источник: houtini.com (2026)

JS-heavy карьерные сайты (Mercedes F1, FIA, McLaren, Audi Motorsport) нельзя scrape обычным HTTP — они рендерятся на клиенте. Firecrawl с browser rendering + structured extract выгребает все вакансии, нормализует в единый формат. Реальный кейс применения в нишевом рекрутинге.

7. 📦 **Community Skill: полный тулkit в одном [SKILL.md](#)**

Источник: lobehub.com (февраль 2026)

GitHub: haniakrim21/everything-claude-code. Готовый скилл с полным набором: single page scrape, full site crawl, batch URL list, site structure map, browser actions (click/scroll/wait для dynamic pages). Реальные use cases в [SKILL.md](#): pricing research, market research, SEO analysis, content migration, test automation, monitoring.

8. 📖 **RAG на живой документации**

Источник: Multiple (eesel.ai guide, Firecrawl docs)

Вместо загрузки PDF-снимотов — crawl официальной документации при каждом запросе. Claude всегда работает с актуальной версией. Особенно критично для быстро меняющихся инструментов (Claude SDK, n8n, etc.). Связка: firecrawl crawl → chunks → pgvector indexing → semantic search.

9. 🏗️ **Open Agent Builder: Firecrawl как нода в pipeline**

Источник: firecrawl.dev/blog/open-agent-builder (ноябрь 2025)

Visual no-code builder где Firecrawl — нода в AI-пайплайне. Цепочки типа: webhook trigger → firecrawl scrape → LLM process → send to Telegram. Для тех кто хочет агентские автоматизации без написания кода. Open-source.

10. 🔍 **Research Pipeline: Search → Scrape → Extract → Synthesize**

Источник: Паттерн из нескольких источников (Firecrawl docs + community)

Полный research agent: firecrawl_search (топ-10 релевантных URL) → scrape каждого → extract структурированных данных → Claude синтезирует единый отчёт. Время: 2-5 минут вместо часа ручного исследования. Применение: due diligence перед партнёрством, конкурентный анализ, технический research перед внедрением.

Источники

Гайд основан на анализе 7 видео за 13 марта 2026:

- [Firecrawl_dev](#) — официальный канал (crawl endpoint, CLI skills, scheduled tasks)
- [Mark_Kashef](#) — MCP setup guide
- [kacperrutk](#) — scraping в 2026, конкурентная разведка
- [MaxMünstermannGTM](#) — лидогенерация без Clay
- [Chase-H-AI](#) — топ-10 Claude Code плагинов

Создано: 2026-03-13 | ClaudeClaw